

Process Synchronization

Ms.swati Mali

Ms. Nirmala Shinde



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

11/11/2024



Process Synchronization

- Processes can execute concurrently but **May be interrupted at any time, partially completing execution**
- Concurrent access to shared data may result in data inconsistency
- Maintaining data consistency requires **mechanisms to ensure the orderly execution of cooperating processes**

Producer Consumer Problem Revisited

- Modifying the Bounded Buffer Approach for Producer and Consumer Problem.
- Having a **counter** that keeps track of the number of full buffers.
 - incremented every time on addition of a new item to the buffer
 - decremented every time on removal of one item from the buffer.

Producer Consumer Problem

- The code for the producer-consumer process can be modified as follows:

```
while (true) {  
    /* produce an item in nextProduced */  
    while (counter == BUFFER_SIZE)  
        ; /* do nothing */  
    buffer[in] = nextProduced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```

```
while (true) {  
    while (counter == 0)  
        ; /* do nothing */  
    nextConsumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
    /* consume the item in nextConsumed */  
}
```

Issue?

- "counter++" may be implemented in machine language as –
 register1 = counter
 register1 = register1 + 1
 counter= register1
- register1 is one of the local CPU registers.
- "counter--" is implemented as follows:
 register2 = counter
 register2 = register2 ~ 1
 counter= register2
- register2 is one of the local CPU registers.

Producer Consumer Problem Revisited

- What if counter and registers are not protected against data inconsistency?

T_0 :	producer	execute	$register_1 = counter$	{ $register_1 = 5$ }
T_1 :	producer	execute	$register_1 = register_1 + 1$	{ $register_1 = 6$ }
T_2 :	consumer	execute	$register_2 = counter$	{ $register_2 = 5$ }
T_3 :	consumer	execute	$register_2 = register_2 - 1$	{ $register_2 = 4$ }
T_4 :	producer	execute	$counter = register_1$	{ $counter = 6$ }
T_5 :	consumer	execute	$counter = register_2$	{ $counter = 4$ }

Producer Consumer Problem Revisited

- If the order of the statements at T4 and T5 is reversed, we would arrive at the incorrect state
 - Result=> "counter == 6", incorrect state

T ₀ :	producer	execute	register ₁ = counter	{register ₁ = 5}
T ₁ :	producer	execute	register ₁ = register ₁ + 1	{register ₁ = 6}
T ₂ :	consumer	execute	register ₂ = counter	{register ₂ = 5}
T ₃ :	consumer	execute	register ₂ = register ₂ - 1	{register ₂ = 4}
T ₄ :	producer	execute	counter = register ₁	{counter = 6}
T ₅ :	consumer	execute	counter = register ₂	{counter = 4}

we allowed both processes to manipulate the variable counter concurrently. – Race Condition



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering



The Critical Section Problem



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

11/11/2024



The Critical Section Problem

- Consider a system consisting of n processes $\{P_0, P_1, \dots, P_{n-1}\}$.

Critical Section-

- Each process has a segment of code, called a critical section in which the process may be
 - changing common variables,
 - updating a table,
 - writing a file, and so on.

The Critical Section Problem

- When one process is executing in its critical section,
 - No other process is to be allowed to execute in its critical section.
 - Execution of Critical Sections by the processes is mutually exclusive
- *Critical-section problem* is to design a protocol that the processes can use to cooperate.

The Critical Section Problem

- **Entry section**

- Each process must request permission to enter its critical section. The section of code implementing this request is the entry section.

- **Exit Section**

- The critical section may be followed by an exit section.

- **Remaining Section**

- The remaining code is the remaining section.

The Critical Section Problem

Solution must Satisfy three requirements-

- 1) Mutual Exclusion**
- 2) Progress**
- 3) Bounded Waiting**

Reading Assignment; why bounded waiting?

- **Bounded Waiting:** Guarantees fairness by limiting wait times.
- **Busy Waiting:** Continuous polling, wastes CPU cycles.
- **Unbounded Waiting:** No guarantees on wait time, leads to starvation.
- **Blocking (Sleep Waiting):** Process sleeps until a condition is met, saves CPU cycles.
- **Starvation:** Indefinite waiting due to unfair resource allocation.
- **Priority Inversion:** High-priority process waits on a lower-priority process.
- **Conditional Waiting:** Waits for a condition to become true.
- **Voluntary Waiting:** A process yields control voluntarily.
- **Preemptive Waiting:** Process is forcibly removed from the CPU.

The Critical Section Problem

- **Mutual Exclusion:** If Process P_i is executing in its critical section, then no other processes can be executing in their critical sections
- **Progress :**
 - Only those processes interested in entering CS , should compete to enter CS.
- **Bounded Waiting :**
 - After a max bound/time limit , the process definitely will get a chance to enter CS

Reading slides: Critical-Section Handling in OS

- Why would anyone favour a preemptive kernel over a non-preemptive one?
- A preemptive kernel
 - is more suitable for real-time programming, as it will allow a real-time process to preempt a process currently running in the kernel.
 - may be more responsive, since there is less risk that a kernel-mode process will run for an arbitrarily long period before relinquishing the processor to waiting processes.

Solutions to Mutual exclusion problems

- **Software support**
- **Hardware support**
- **Support from Programming languages**
- **Support from OS**

solution mechanisms to achieve concurrency

- **Software approach:**

- Here no hardware, OS, or programming language level supported is assumed.
- Burdens programmers to write the programs (which when executed becomes processes) to achieve concurrency themselves.
- Dekker's or Peterson's algorithms are known for the software approach solutions to mutual exclusion.

- **Hardware approach :**

- Use of special hardware instructions and control over the hardware such as disabling the interrupts.

- **Support from operating systems and programming languages:**

- Here, the operating system offers support to some signals
- the programmers can be provided with language constructs or library with which they can easily write the code for the process synchronize with each other.

Requirements for mutual exclusion

- Mutual exclusion must be enforced: Only one process at a time is allowed into its critical section
- A process that halts in its noncritical section must do so without interfering with other processes.
- The solution must ensure that there is no deadlock or starvation.
- When no process is in a critical section, any process that requests entry to its critical section must be permitted to enter without delay.
- No assumptions are made about relative process speeds or number of processors.
- A process remains inside its critical section for a finite time only
- Mutual exclusion techniques can be used to resolve conflicts, such as competition for resources, and to synchronize processes so that they can cooperate

Illustration of Mutual Exclusion

<pre>/* PROCESS 1 */ void P1 { while (true) { /* preceding code */; EnterCriticalSection (Ra); /* Execute Critical Section */; ExitCriticalSection (Ra); /* remaining code */; } }</pre>	<pre>/* PROCESS 2 */ void P2 { while (true) { /* preceding code */; EnterCriticalSection (Ra); /* Execute Critical Section */; ExitCriticalSection (Ra); /* remaining code */; } }</pre>	...	<pre>/* PROCESS n */ void Pn { while (true) { /* preceding code */; EnterCriticalSection (Ra); /* Execute Critical Section */; ExitCriticalSection (Ra); /* remaining code */; } }</pre>
---	---	------------	---

Software support



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

11/11/2024



Reading Assignment

Attempt One

```
/* Process 0*/  
.  
.  
.  
while(turn!=0)  
/* do nothing */;  
/*critical section*/;  
turn =1;  
.  
.
```

```
/* Process 1*/
```

```
.  
.  
.  
while(turn!=1)  
/* do nothing */;  
/*critical section*/;  
turn =0;  
.  
.
```

Attempt Two

```
/* Process 0*/  
.  
.  
.  
while(flag[1])  
/* do nothing */;  
flag[0]=true;  
/*critical section*/;  
flag[0]=false;  
.  
.
```

```
/* Process 1*/
```

```
.  
.  
.  
while(flag[0])  
/* do nothing */;  
flag[1]=true;  
/*critical section*/;  
flag[1]=false;  
.  
.
```

First Attempt:

- Succeeds in enforcing mutual exclusion
- Uses variable to control which thread can execute
- Constantly tests whether critical section is available
 - Busy waiting
 - Wastes significant processor time
- Problem known as strict alternation synchronization
 - Each thread can execute only in strict alternation

Second Attempt:

- Removes strict alternation synchronization
- Violates mutual exclusion
 - Thread could be preempted while updating flag variable
 - Not an appropriate solution

Reading Assignment

Attempt Three		Attempt Four	
<pre>/* Process 0*/ . . . flag[0]=true; while(flag[1]) /* do nothing */; /*critical section*/; flag[0]=false; ; . .</pre>	<pre>/* Process 1*/ . . . flag[1]=true; while(flag[0]) /* do nothing */; /*critical section*/; flag[1]=false; ; . .</pre>	<pre>/* Process 0*/ . . . flag[0]=true; while(flag[1]) { flag[0]=false; /*delay*/ flag[0]=true; } /*critical section*/; flag[0]=false; . .</pre>	<pre>/* Process 1*/ . . . flag[1]=true; while(flag[0]) { flag[1]=false; /*delay*/ flag[1]=true; } /*critical section*/; flag[1]=false; . .</pre>

Third Attempt:

- Set critical section flag before entering critical section test
 - Once again guarantees mutual exclusion
- Introduces possibility of deadlock
 - Both threads could set flag simultaneously
 - Neither would ever be able to break out of loop
- Not a solution to the mutual exclusion

Fourth Attempt:

- Sets flag to false for small periods of time to yield control
- Solves previous problems, introduces indefinite postponement
 - Both threads could set flags to same values at same time
 - Would require both threads to execute in tandem (unlikely but possible)
- Unacceptable in mission- or business-critical systems

Correct solutions: Dekker's solution

```
boolean flag [2];
int turn;
```

```
void main()
{ flag[0]=flag[1]=false;
  turn=1;
  parbegin(P0,P1);
}
```

```
void P0()
{
  while (true)
  {
    flag[0]=true;
    while (flag[1])
      if (turn==1)
      {
        flag[0]=false;
        while (turn==1)
          /* Do
            Nothing*/
        flag[0]=true;
      }
    /*Critical Section */
    turn=1;
    flag[0]=false;
    /*Remainder Code*/
  }
}
```

```
void P1()
{
  while (true)
  {
    flag[1]=true;
    while (flag[0])
      if (turn==0)
      {
        flag[1]=false;
        while (turn==0)
          /* Do
            Nothing*/
        flag[1]=true;
      }
    /*Critical Section */
    turn=0;
    flag[1]=false;
    /*Remainder Code*/
  }
}
```

The characteristics:

- Proper solution
- Guarantees mutual exclusion
- Avoids previous problems of deadlock, indefinite postponement
- Uses notion of favored threads to determine entry into critical sections
 - Resolves conflict over which thread should execute first
 - Each thread temporarily unsets critical section request flag
 - Favored status alternates between threads
- wasn't accepted as a solution to mutual exclusion as it could support only two processes and in the real scenario system has many processes to handle.

Correct solutions: Peterson's solution

```
void Pi()  
{  
    While (true)  
    {  
        flag[i] := true;  
        turn := j;  
        while (flag[ j] and turn  
= j )  
            /*Do Nothing*/ ;  
        /*Critical Section */  
        flag[i] := false;  
        /*Remainder Section*/  
    }  
}
```

```
boolean flag[n];  
Int turn;  
  
void main()  
{  
    flag[i] := false;  
    flag[ j] := false;  
  
    parbegin(Pi,Pj,....Pn)  
    ;  
}
```

The characteristics:

- Proper solution
- Guarantees mutual exclusion
- Avoids problems of deadlock, indefinite postponement
- Uses notion of favored threads to determine entry into critical sections
 - Resolves conflict over which thread should execute first
 - Each thread temporarily unsets critical section request flag
 - Favored status alternates between threads
 - Can be applied to any number of processes

Expected outcome of reading assignment:

Detect Livelock

Detect Deadlock

Detect bounded waiting

Detect Progress



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

11/11/2024



Livelock situation while inserting a node in BST

Process P1:

while True:

 Lock tree for insertion

 if P2 is waiting to insert a node:

 Release tree lock

 Wait for P2 to finish its
 operation

 else:

 Insert node A

 Release tree lock

 break

Exit loop when insertion is successful

Process P2:

while True:

 Lock tree for insertion

 if P1 is waiting to insert a node:

 Release tree lock

 Wait for P1 to finish its
 operation

 else:

 Insert node B

 Release tree lock

 break

Exit loop when insertion is successful

Deadlock situation

Process P1:

Lock R1

Wait(100 ms) # some work with R1

Lock R2 # Trying to lock R2

(but P2 may have already locked R2)

// Critical section using R1 and R2

Unlock R2

Unlock R1

Process P2:

Lock R2

Wait(100 ms) # some work with R2

Lock R1 # Trying to lock R1

(but P1 may have already locked R1)

// Critical section using R2 and R1

Unlock R1

Unlock R2

Busywait situation while inserting a node in BST

Shared Variable:

```
is_inserting = False // Flag to indicate if insertion is in progress
```

Algorithm InsertNode(root, key):

```
while is_inserting == True: // Busy waiting loop
    // Do nothing, just wait until is_inserting becomes False
    continue
    // Set is_inserting to True to indicate this process is inserting
is_inserting = True
if root is NULL:
    root = CreateNewNode(key)
else if key < root.value:
    root.left = InsertNode(root.left, key)
else if key > root.value:
    root.right = InsertNode(root.right, key)
// Insertion is done, set is_inserting to False
is_inserting = False
```

```
return root
```

Hardware Synchronization



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

11/11/2024



Hardware approaches to enforce mutual exclusion

- **Disabling interrupt** and
- **Use of special instructions.**

Disabling interrupt

- In a uniprocessor system, concurrent processes are not overlapped but they can only be interleaved.
- Furthermore, a process executes until it calls an OS service or until it is interrupted.
- Thus, it is sufficient to prevent a process from being interrupted to guarantee mutual exclusion, This capability can be provided in the form of primitives defined by the OS kernel for disabling and enabling interrupts.

Disabling interrupt

The pseudo code:

```
while (true) {  
    /* disable interrupts */;  
    /* critical section */;  
    /* enable interrupts */;  
    /* remainder */;  
}
```

Analysis: Disabling interrupt

- As the processes are not interrupted, the mutual exclusion is guaranteed.
- seems simple, the price paid for the same is very high.
- As the processor is not allowed to interleave the processes, the efficiency and overall system performance degrades significantly.
- Also, this particular solution will not work with multiprocessor architecture.
- When the computer includes more than one processor, it is possible (and typical) for more than one process to be executing at a time. In this case, disabled interrupts do not guarantee mutual exclusion.

Special Machine Instructions:

- In a multiprocessor systems, several processors share main memory.
- Moreover, the interrupt disabling cannot help the mutual exclusion.
- Special Atomic Hardware Instructions-
 - Atomic = Non-Interruptible
- So to give mutually exclusive access to any memory location the processor designers have proposed several machine instructions that carry out some actions atomically, with one instruction fetch cycle.
- Examples: compare&Swap, test&set, Exchange, etc

Special Machine Instructions: Compare & Swap

Compare & Swap Instruction

```
int compare_and_swap (int *word, int testvalue, int newvalue)
{
    int oldvalue;
    oldvalue = *word;
    if (oldvalue == testvalue) *word = newvalue;
    return oldvalue;
}
```

test_and_set Instruction

Definition:

```
boolean test_and_set (boolean *target)
{
    boolean rv = *target;
    *target = TRUE;
    return rv;
}
```

- Executed atomically
 - If 2 test and set instructions are executed simultaneously , they will be executed sequentially in some arbitrary order.
- Returns the original value of passed parameter i.e False
- Set the new value of passed parameter to “TRUE”.

test_and_set Instruction

Definition:

```
boolean test_and_set (boolean *target)
{
    boolean rv = *target;
    *target = TRUE;
    return rv;
}
```

- Executed atomically
 - If 2 test and set instructions are executed simultaneously , they will be executed sequentially in some arbitrary order.
- Returns the original value of passed parameter i.e False
- Set the new value of passed parameter to “TRUE”.

Mutual-exclusion implementation with TestAndSet ()

P0

```
do{  
  
while(TestAndSet(&lock));  
    critical section  
    lock=FALSE;  
    remainder section  
}while(1);
```

- P0 tries to enter CS
- T&S returns False and set lock=True
- While(False);
- P0 comes out while
- Enter CS

- Now P1 tries to enter CS
- P0 is still inside CS and lock=true
- Test&Set Returns the original value of passed parameter i.e. True
- While(True);
- Goes in Infinte Loop/Do Nothing
- so P1 cannot enter CS

Mutual-exclusion implementation with TestAndSet ()

P0

```
do{  
  
while(TestAndSet(&lock));  
    critical section  
    lock=FALSE;  
    remainder section  
}while(1);
```

- After completing CS
- P0 makes lock=false
- Exits CS
- Enters RS

- Now P1 can enter CS as lock is false now
- Comes out of while loop
- Executes CS

swap Instruction

Definition:

```
void Swap(boolean *a, boolean *b) {  
    boolean temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

- Swaps the contents of 2 memory word
- Executed atomically=Non-interruptable

Mutual-exclusion implementation with swap ()

- Solution:

```
do {  
    key = TRUE;  
    while (key == TRUE)  
        Swap(&lock, &key);  
  
    // critical section  
  
    lock = FALSE;  
  
    // remainder section  
} while (TRUE);
```

Mutual-exclusion implementation with the Swap () instruction.

- Key = Local Variable = Each process has its own

= Global Variable = Common for all
processes

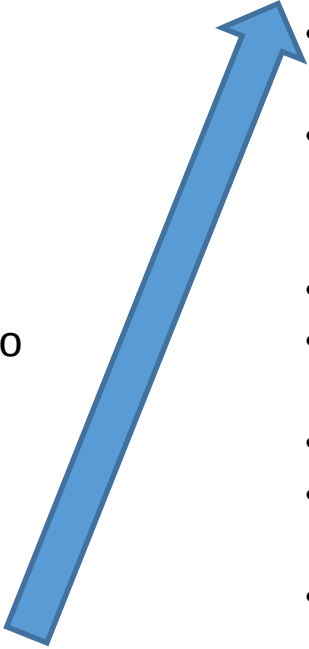
Initialized to False

Mutual-exclusion implementation with swap ()

- **Solution:**

```
do {  
    key = TRUE;  
    while (key == TRUE)  
        Swap(&lock, &key);  
  
    // critical section  
  
    lock = FALSE;  
  
    // remainder section  
} while (TRUE);
```

Mutual-exclusion implementation with the Swap () instruction.

- Initially (L,K)=(F,F)
 - P0 makes key=true so
 - (L,K)=(F,T)
 - While key is true
 - Swap values of Key and Lock so
 - (L,K)=(T,F)
 - Key becomes false, exits while loop
 - Enters CS
 - Now P1 tries to enter CS
 - P1's own key is False Initially
 - Global Lock is True as P0 is in CS
 - Key=true so
 - (L,K)=(T,T)
 - After Swapping also,
 - Key will always be True, So trapped in while loop
 - P1 will not be able to enter CS
- 

Mutual-exclusion implementation with swap ()

- **Solution:**

```
do {  
    key = TRUE;  
    while (key == TRUE)  
        Swap(&lock, &key);  
  
    // critical section  
  
    lock = FALSE;  
  
    // remainder section  
} while (TRUE);
```

Mutual-exclusion implementation with the Swap () instruction.

- P0
- Now, Lock=False and P0 exits CS
- P0 enters RS



- Now P1 tries to enter CS
- So Pair (L,K)=(F,T)
- The value will be swapped
- Key will become False
- Pair(L,K)=(T,F)
- Control comes out of while
- Enters CS

Reading Assignment: Hardware Synchronization

- 1) Test Memory word and Set value-Test and Set()
- 2) Swap contents of two memory words-Swap()

These algorithms do not satisfy the bounded waiting requirement

Reading Assignment: Bounded Waiting implementation with test and set ()

- Solution:

```
do {  
    waiting[i] = TRUE;  
    key = TRUE;  
    while (waiting[i] && key)  
        key = TestAndSet(&lock);  
    waiting[i] = FALSE;
```

→ Entry Code

```
    // critical section  
  
    j = (i + 1) % n;  
    while ((j != i) && !waiting[j])  
        j = (j + 1) % n;  
  
    if (j == i)  
        lock = FALSE;  
    else  
        waiting[j] = FALSE;  
  
    // remainder section  
} while (TRUE);
```

→ Exit Code

Reading Assignment: Bounded Waiting implementation with test and set ()

```
do {
    waiting[i] = TRUE;
    key = TRUE;
    while (waiting[i] && key)
        key = TestAndSet(&lock);
    waiting[i] = FALSE;

    // critical section

    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;

    if (j == i)
        lock = FALSE;
    else
        waiting[j] = FALSE;

    // remainder section
} while (TRUE);
```

- Solution: Satisfies all 3 critical section requirements
- Shared Data structures/Global variables
 - boolean waiting[n];
 - boolean lock;
- Both initialized to false

Reading Assignment: Bounded Waiting implementation with test and set ()

```
do {
    waiting[i] = TRUE;
    key = TRUE;
    while (waiting[i] && key)
        key = TestAndSet(&lock);
    waiting[i] = FALSE;

    // critical section

    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;

    if (j == i)
        lock = FALSE;
    else
        waiting[j] = FALSE;

    // remainder section
} while (TRUE);
```

- Lock is initialized to False
- P_i can enter its critical section only if either $waiting[i] == false$ or $key == false$
- The value of Key can become false only if Test&Set() is executed
- The 1st process to execute the Test&Set() will set $key == false$ and lock to True
- $waiting[i]$ of $P_i = False$
- All other processes must wait

Reading Assignment: Bounded Waiting implementation with test and set ()

```
do {
    waiting[i] = TRUE;
    key = TRUE;
    while (waiting[i] && key)
        key = TestAndSet(&lock);
    waiting[i] = FALSE;

    // critical section

    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;

    if (j == i)
        lock = FALSE;
    else
        waiting[j] = FALSE;

    // remainder section
} while (TRUE);
```

- Lock=false
- For P_i , waiting and Key both are True
- while Condtn=True
 - Inside While Loop,
 - Key=T&S(&Lock)
 - So Key=False(Returns Original Value of Lock=False)
 - Lock=True
- exit form While loop as Key=false
- Waiting of P_i finishes=false
- Enters Critical Section
- Now P_j tries to enter Critical Section
- waiting and key Both are True,
- Lock=True
- while conditn=True
 - Inside While Loop
 - Key=T&S(&Lock)
 - Key=True
 - Lock=True
 - Trapped in while loop

Reading Assignment: Bounded Waiting implementation with test and set ()

- The hardware-based solutions to the CSP are complicated for application programmers to use.
- Soln= Semaphore

Advantages of hardware approaches to enforce mutual exclusion.

- The solution works with any number of processes on uniprocessor as well as multiprocessors architectures.
- It is very simple and therefore easy to verify.
- It can also support multiple critical sections; and each critical section can be defined by its own variable.

Disadvantages of hardware approaches to enforce mutual exclusion.

- o **Busy waiting is employed.**
- o **Starvation is possible.**
- o **Deadlock is possible.**
- o Consider the following scenario on a single-processor system.
 - o Process P1 executes the special instruction (e.g., compare&swap, exchange) and enters its critical section.
 - o P1 is then interrupted to give the processor to P2, which has higher priority.
 - o If P2 now attempts to use the same resource as P1, it will be denied access because of the mutual exclusion mechanism.
 - o Thus it will go into a busy waiting loop.
 - o However, P1 will never be dispatched because it is of lower priority than another ready process, P2.
 - o Because of the drawbacks of both the software and hardware solutions just outlined, we need to look for other mechanisms

Semaphores

Semaphore

- **Semaphore:** Semaphore is an integer variable that can be initialized to some value.
- The semaphore values are used for signaling the processes for communication.
- The semaphores can be operated upon by three operations viz, initialize, increment and decrement.
- All these operations are atomic
- Depending on exact definition of semaphore, the decrement (wait) operation may block a process and the increment (signal) operation may unblock a process.
- Semaphores are supported by operating system to achieve concurrency and synchronization.

Semaphores : Types of semaphore

- **Binary Semaphore:**

- Binary Semaphore is a semaphore that takes on only the values 0 and 1.

- **General or counting Semaphore:**

- Any Semaphore that is not restricted to have value only 0 and 1 and can have values ≥ 2 are called general or counting semaphores.

- In other words, the nonbinary semaphore is often referred to as either a counting semaphore or a general semaphore.

- **Strong Semaphore:**

- A semaphore whose definition includes FCFS policy when it is unblocked from the blocked queue, is called a Strong Semaphore.

- **Weak Semaphore:**

- A semaphore whose definition does not include any policy when it is unblocked from the blocked queue, is called a Weak Semaphore.

Mutex

- Short for **mutual exclusion object**.
- A mutex is a program object that allows multiple program threads to share the same resource, such as file access, but not simultaneously.
- A mutex object only allows one thread into a controlled section, forcing other threads which attempt to gain access to that section to wait until the first thread has exited from that section.
- When a program is started, a mutex is created with a unique name.
- After this stage, any thread that needs the resource must lock the mutex from other threads while it is using the resource.
- The mutex is set to unlock when the data is no longer needed or the routine is finished.

Mutex and Binary Semaphores

- **Mutex** is a specialized form of binary semaphore designed specifically for mutual exclusion with the added restriction that only the thread that locks the mutex can unlock it.
- **Binary Semaphore** is more general and can be used for signaling and synchronization, not necessarily tied to ownership of resources.

Mutex and Binary Semaphores

Aspect	Mutex	Binary Semaphore
Ownership	A mutex can only be released (unlocked) by the thread that acquired (locked) it.	A binary semaphore can be signaled (released) by any thread, even one that didn't wait (acquire) it.
Use Case	Typically used for locking a resource or critical section, where ownership is important.	Typically used for signaling between processes or threads, not necessarily for resource locking.
Purpose	Specifically designed for mutual exclusion.	Designed for signaling (can be used for both synchronization and mutual exclusion).
Overhead	Generally has lower overhead since it's specifically designed for locking mechanisms.	Slightly higher overhead due to its more general-purpose use in synchronization.
Priority Inversion Handling	Mutexes often come with built-in mechanisms to prevent priority inversion (e.g., priority inheritance).	Binary semaphores do not typically have built-in mechanisms to handle priority inversion.
Typical API	lock() and unlock().	wait() (or P()) and signal() (or V()).

semaphore primitives

- Semaphore is an integer variable that can be initialized to some value.
- The semaphore values are used for signaling the processes for communication.
- They can be operated upon by three operations viz,
 - initialize,
 - increment and
 - decrement.
- All these operations are atomic in nature.
- Depending on exact definition of semaphore, the decrement (wait) operation may block a process and the increment (signal) operation may unblock a process.

Definition of Semaphore Primitives

```
struct semaphore {  
  int count;  
  queueType queue;  
};
```

```
void semWait(semaphore s)  
{  
  s.count--;  
  if (s.count < 0) {  
    /* place this process in s.queue */;  
    /* block this process */;  
  }  
}
```

```
void semSignal(semaphore s)  
{  
  s.count++;  
  if (s.count <= 0) {  
    /* remove a process P from s.queue */;  
    /* place process P on ready list */;  
  }  
}
```

```
/* program mutualexclusion */

const int n = /* number of processes */;

semaphore s = 1;
void P(int i)
{
    while (true) {
        semWait(s);
        /* critical section */;
        semSignal(s);
        /* remainder */;
    }
}

void main()
{
    parbegin (P(1), P(2), . . ., P(n));
}
```

Binary Semaphore

Mutual Exclusion using Binary Semaphore

- Used to deal with the critical-section problem for multiple processes.
- Processes share a semaphore, mutex, initialized to 1.

```
do {  
    wait(mutex);  
  
    // critical section  
  
    signal(mutex);  
  
    // remainder section  
} while (TRUE);
```

```
wait(S) {  
    while S <= 0  
    ; // no-op  
    S--;  
}
```

Mutual Exclusion using Binary Semaphore

- Process P0 tries to enter the CS

P0

while(mutex<=0);

mutex=1 so Condition is False

Comes out of while loop

mutex - -

so mutex=0

```
do {  
    wait(mutex);  
  
    // critical section  
  
    signal(mutex);  
  
    // remainder section  
} while (TRUE);
```

```
wait(S) {  
    while S <= 0  
    ; // no-op  
    S--;  
}
```

Now P1 tries to enter CS

while(mutex<=0);

condition is True , so P1 gets trapped in a Do Nothing Loop

P1 cannot enter CS

=====>ME preserved

Semaphore

- We can also use semaphores to solve various synchronization problems.
- Consider two concurrently running processes:
 - P1 with a statement S1
 - P2 with a statement S2 .
 - Suppose we require that S2 be executed only after S1 has completed.

Semaphore

- Let P1 and P2 share a **common semaphore synch**, initialized to 0

- Statements inserted in P1

S1;
signal(synch);

- Statements inserted in P2

wait(synch);
S2;



- Because synch is initialized to 0,
- P2 will execute S2 only after P1 has invoked signal (synch), which is after statement S1 has been executed.
 - **Else P2 will caught in infinte loop inside wait() fn**

```
wait(S) {  
  while S <= 0  
  ; // no-op  
  S--;  
}
```

ISRO | ISRO CS 2017 – May | Question 78

At particular time, the value of a counting semaphore is 10, it will become 7 after:

- (a) 3 V operations
- (b) 3 P operations
- (c) 5 V operations and 2 P operations
- (d) 2 V operations and 5 P operations

Which of the following option is correct?

- (A) Only (b)
- (B) Only (d)
- (C) Both (b) and (d)
- (D) None of these

wait () operation = originally termed P
from the Dutch *proberen*,
Meaning "to test" or "to
attempt"

signal() operation = originally called V
from *verhogen*,
Meaning "to increment"

At particular time, the value of a counting semaphore is 10, it will become 7 after:

- (a) 3 V operations
- (b) 3 P operations
- (c) 5 V operations and 2 P operations
- (d) 2 V operations and 5 P operations

Which of the following option is correct?

- (A) Only (b)
- (B) Only(d)
- (C) Both (b) and (d)
- (D) None of these

Answer: (C)

Explanation: P: Wait operation decrements the value of the counting semaphore by 1.

V: Signal operation increments the value of counting semaphore by 1.

Current value of the counting semaphore = 10

a) after 3 P operations, value of semaphore = $10 - 3 = 7$

d) after 2 v operations, and 5 operations value of semaphore = $10 + 2 - 5 = 7$

Hence option (C) is correct.

UGC-NET | UGC NET CS 2018 July – II | Question 51

At a particular time of computation, the value of a counting semaphore is 10. Then 12 P operations and “x” V operations were performed on this semaphore. If the final value of semaphore is 7, x will be:

- (A) 8
- (B) 9
- (C) 10
- (D) 11

Answer: (B)

Explanation: Initially the value of a counting semaphore is 10. Now 12 P operations are performed.

Now counting semaphore value = -2

“x” V operations were performed on this semaphore and final value of counting semaphore = 7

i.e $x + (-2) = 7$

$x = 9$.

So, option (B) is correct.

Semaphore Definition

- With each semaphore there is an associated waiting queue
- Each entry in a waiting queue has two data items:
 - value (of type integer)
 - pointer to next record in the list

```
typedef struct {  
    int value;  
    struct process *list;  
} semaphore;
```

Semaphore Definition

- Wait()
 - When a process must wait on a semaphore, it is **added to the list of processes**.
- Signal()
 - A signal() operation **removes one process from the list** of waiting processes and awakens that process

```
typedef struct {  
    int value;  
    struct process *list;  
} semaphore;
```

Implementation with no busy waiting

- We modify the definition of the wait() and signal() semaphore operations.
- When a process executes the wait () operation and finds that the semaphore value is not positive, it must wait.
- Rather than busy waiting, the process can *block* itself.

Implementation with no busy waiting

- Two operations:
 - **block**
 - **wakeup**

Implementation with no busy waiting

- Two operations:
 - **block** –
 - The block() operation suspends the process that invokes it.
 - **place the process** invoking the operation **on the waiting queue** associated with the semaphore
 - The state of the process is switched to the **waiting state**.
 - Control transferred to the **CPU scheduler**, which selects another process to execute.

Implementation with no busy waiting

wakeup –

- A process that is blocked, waiting on a semaphore S, **should be restarted** when some **other process executes a signal()** operation.
- The process is restarted by a wakeup () operation

Implementation with no busy waiting

wakeup –

- The wakeup(P) operation **resumes the execution** of a blocked process P.
- Remove one of processes in the waiting queue and place it in the ready queue
- Changes the process state from **waiting to ready**
- These two operations are provided by the operating system as basic system calls.

Implementation with no busy waiting

```
wait(semaphore *S) {  
    S->value--;  
    if (S->value < 0) {  
        add this process to S->list;  
        block();  
    }  
}
```

```
signal(semaphore *S) {  
    S->value++;  
    if (S->value <= 0) {  
        remove a process P from S->list;  
        wakeup(P);  
    }  
}
```

Implementation with no busy waiting

- Semaphore values **may be negative**,
- If a semaphore value is negative, its magnitude is the **no of processes waiting on that semaphore**.
- Semaphore values are **never negative under the classical definition** of semaphores with busy waiting.

Implementation with no busy waiting

- The list of waiting processes can be easily implemented by
 - a link field in each process control block (PCB).
- Each semaphore contains
 - an integer value and
 - a pointer to a list of PCBs.
- One way to add and remove processes from the list
 - so as to ensure bounded waiting is to use a FIFO queue,
 - where the semaphore contains both head and tail pointers to the queue.
 - In general, the list can use *any* queueing strategy.

Dining Philosophers problem

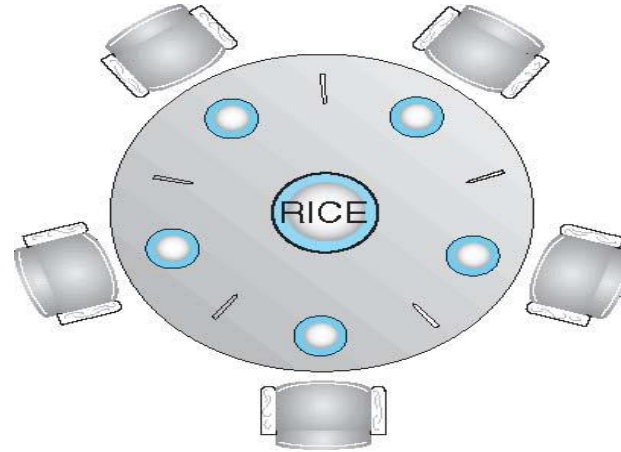
A solution of semaphores

Dining-Philosophers Problem



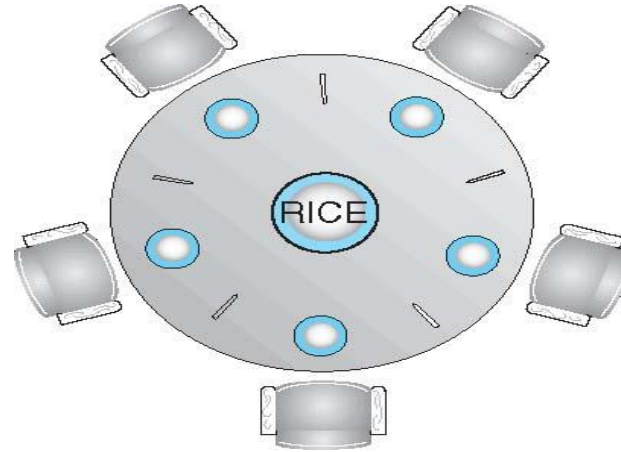
- Philosophers spend their lives alternating thinking and eating
- When thinking
 - Don't interact with their neighbors
- When hungry-
 - A philosopher needs both their right and left chopstick to eat.
 - A hungry philosopher may only eat if there are both chopsticks available

Dining-Philosophers Problem



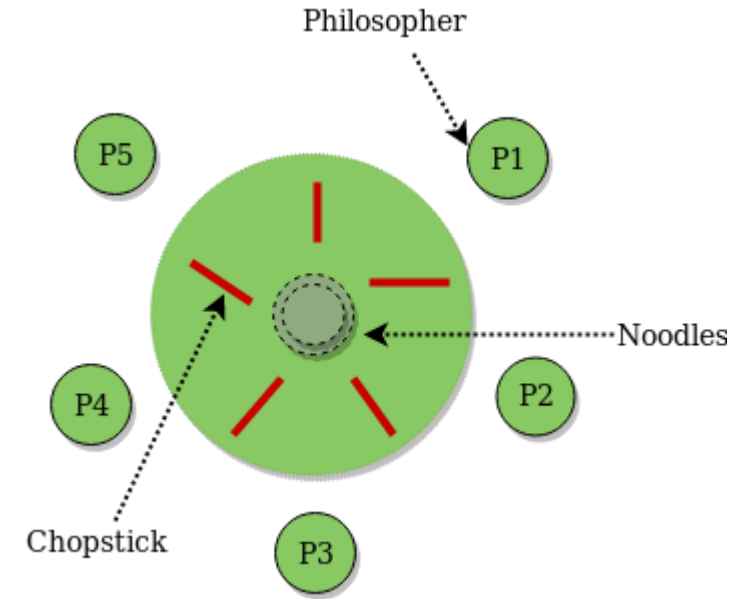
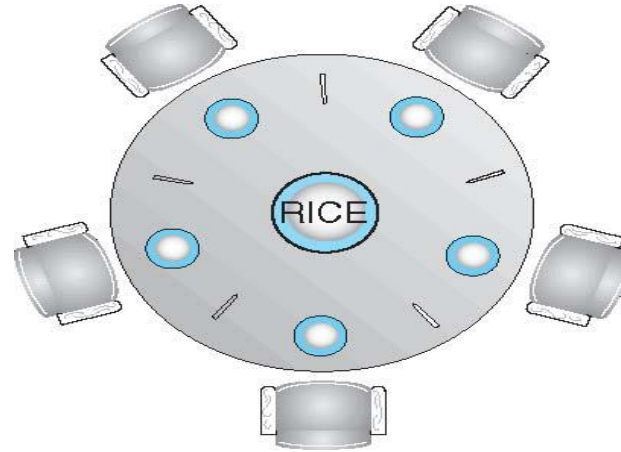
- Occasionally try to pick up 2 chopsticks that are closest to her to eat from bowl
 - Chopsticks that are between her and her left and right neighbor
 - Pick up only one at a time
 - Need both to eat, then release both when done

Dining-Philosophers Problem



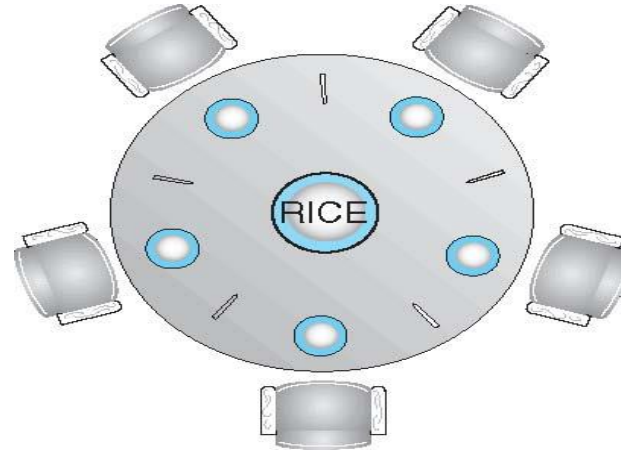
- Classic Synchronization problem
- Example of large class of concurrency control problems
- Represents the need to allocate several resources among several processes
 - in a deadlock free and starvation free manner

Dining-Philosophers Problem



- 5 Philosophers share a common circular table
 - Surrounded by 5 chairs=each belonging to one philosopher
 - Center of table -> Bowl of rice
 - Five single chopsticks
 - Shared data
 - Bowl of rice (data set)
- Semaphore **chopstick** [5] initialized to 1

Semaphore Solution for Dining-Philosophers Problem

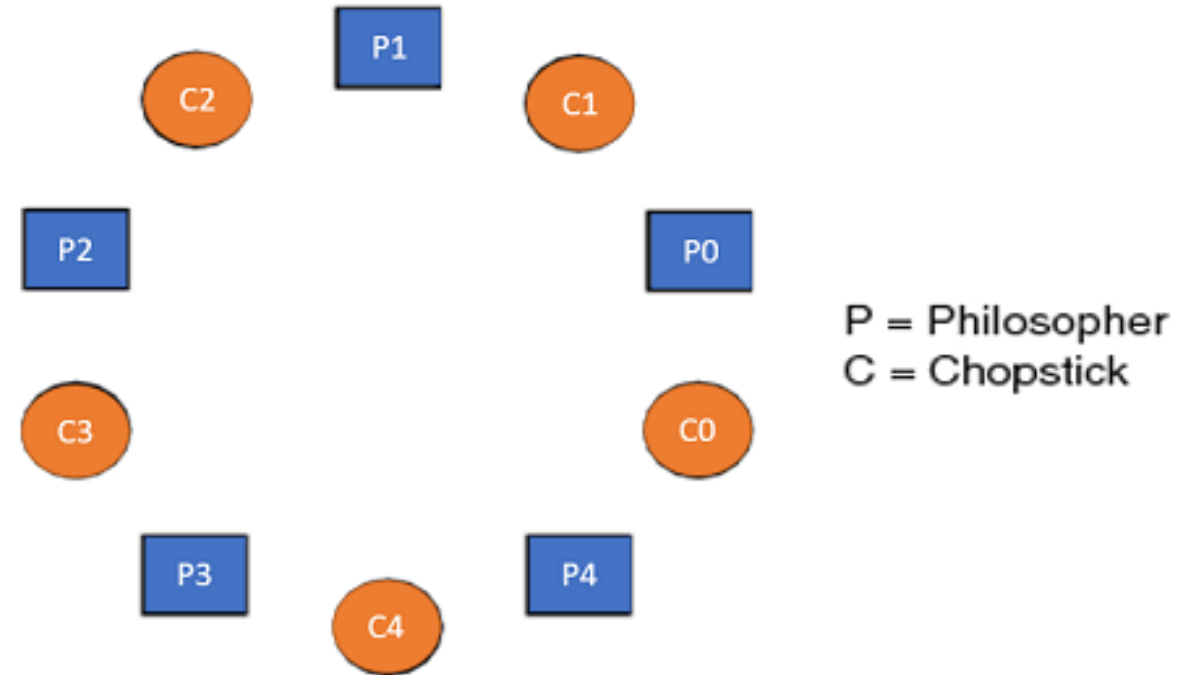


- Grab the chopstick –By Executing wait operation on the semaphore
- Release the chopstick-By executing the signal operation on the appropriate semaphore
- Semaphore **chopstick** [5] initialized to 1

Semaphore Solution for Dining-Philosophers Problem Algorithm

- The structure of Philosopher i :

```
do {  
    wait (chopstick[i] );  
    wait (chopstick[ (i + 1) % 5] );  
  
    // eat  
  
    signal (chopstick[i] );  
    signal (chopstick[ (i + 1) % 5] );  
  
    // think  
  
} while (TRUE);
```



Semaphore Solution for Dining-Philosophers Problem Algorithm

What is the problem with this algorithm?

Semaphore Solution for Dining-Philosophers Problem Algorithm

What is the problem with this algorithm?

- Algorithm Guarantees that No two neighbors are eating simultaneously

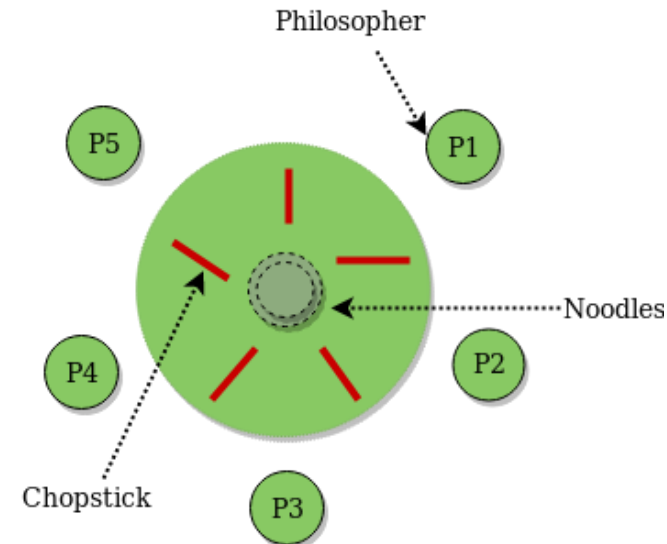
Still Must be rejected

- Why?

Semaphore Solution for Dining-Philosophers Problem Algorithm

Why?

- Possibility of deadlock
- If all 5 philosophers are hungry simultaneously and each grabs left chopstick
- All elements of chopsticks will be $=0$
- When each philosopher tries to grab her right chopstick, delayed forever



Dining Philosopher's problem

- **Assumptions:**
- There are 5 philosophers (can be generalized to any number).
- Philosophers alternate between thinking and eating.
- Philosophers must pick up two chopsticks (left and right) to eat.
- A semaphore controls access to chopsticks.

```

#define N 5                // Number of philosophers

semaphore chopstick[N];    // Semaphores
representing chopsticks (binary semaphores)

// Philosopher function representing actions of each philosopher
void philosopher(int i) {
    while (true) {
        think();           // Think , non-CS activity
        // Pick up left chopstick (chopstick[i])
        wait(chopstick[i]);
        // Pick up right chopstick (chopstick[(i+1)%N])
        wait(chopstick[(i + 1) % N]);
        eat();              // Eat – CS activity
        // Put down right chopstick
        signal(chopstick[(i + 1) % N]);
        // Put down left chopstick
        signal(chopstick[i]);
    }
}

```

```

int main() {
    // Initialize all semaphores to 1 (available chopsticks)
    for (int i = 0; i < N; i++) {
        init_semaphore(chopstick[i], 1);
    }

    // Create threads/processes for each philosopher
    for (int i = 0; i < N; i++) {
        create_thread(philosopher, i);
    }

    // Join threads if needed (depends on implementation)
    join_threads();
    return 0;
}

```

Analysis

- **Semaphores** (chopstick[i]): Each chopstick is represented by a semaphore initialized to 1, indicating that it is available.
- **wait()**: A philosopher waits for the chopstick (decrements the semaphore).
- **signal()**: After eating, a philosopher releases the chopstick (increments the semaphore).
- **Thinking**: Simulated by think().
- **Eating**: Simulated by eat().

Analysis

- Mutual Exclusion?
- Livelock / progress?
- Deadlock?
 - Possible if all philosophers pick up left chopstick and wait for right chopstick, creating circular wait and hold and wait.
- Starvation?
 - a philosopher might wait indefinitely if others repeatedly eat before it.

Semaphore Solution for Dining-Philosophers Problem Algorithm

- Deadlock handling
 - Allow at most 4 philosophers to be sitting simultaneously at the table.
 - Allow a philosopher to pick up the forks only if both are available (picking must be done in a critical section)
 - Use an asymmetric solution --
 - Odd-numbered philosopher picks up first the left chopstick and then the right chopstick.
 - Even-numbered philosopher picks up first the right chopstick and then the left chopstick

```

// Number of philosophers
#define N 5

semaphore chopstick[N];          // Semaphores representing chopsticks (binary
semaphores)

// counting semaphore 'limit' to ensure only N-1 philosophers can access chopsticks at a
time

semaphore limit;

void philosopher(int i) {
    while (true) {
        think();                  // Think : non-CS activity
        // Wait for permission to access chopsticks (only N-1 philosophers can proceed)
        wait(limit);
        wait(chopstick[i]);        // Pick up left chopstick (chopstick[i])
        wait(chopstick[(i + 1) % N]); // Pick up right chopstick (chopstick[(i+1)%N])
        eat();                     //process
        signal(chopstick[(i + 1) % N]); // Put down right chopstick
        signal(chopstick[i]);       // Put down left chopstick
        // Signal that philosopher is done eating, allowing another philosopher to proceed
        signal(limit);
    }
}

```

```

int main() {
    // Initialize all chopsticks to 1 (available)
    for (int i = 0; i < N; i++) {
        init_semaphore(chopstick[i], 1);
    }

    // Initialize mutex to N-1 to allow only N-1 philosophers
    to pick up chopsticks at a time
    init_semaphore(limit, N-1);

    // Create threads/processes for each philosopher
    for (int i = 0; i < N; i++) {
        create_thread(philosopher, i);
    }

    // Join threads if needed (depends on implementation)
    join_threads();

    return 0;
}

```



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

Solution II - deadlock-free, starvation-free, and livelock-free



Analysis

- **Preventing Deadlock:**
 - central *semaphore mutex* (or binary semaphore) ensures that philosophers can only pick up chopsticks if both are available.
- **Preventing Starvation:**
 - central *semaphore* (mutex) allows only N-1 philosophers to pick up chopsticks at a time.
 - Ensures philosophers take turns eating, and no philosopher can indefinitely hold chopsticks if others are waiting.
 - This ensures that there will always be at least one philosopher who can eat.
- **Achieving Mutual Exclusion:**
 - Each chopstick is still protected by a semaphore, ensuring **mutual exclusion**. Only one philosopher can hold a chopstick at a time.
- **Preventing Livelock:**
 - Philosophers will only proceed when they have both chopsticks, and the introduction of mutex ensures that the system does not get stuck in an endless cycle of picking up and putting down chopsticks.

Readers-Writers Problem

Readers-Writers Problem

- Process Synchronization Problem
- A Database/Data set/Object/File/Record is shared among a number of concurrent processes
 - Readers – **only read** the data set; they do **not** perform any updates
 - Writers – **can both read and write i.e.** update

Readers-Writers Problem

Problem –

- Allow multiple readers to read at the same time, no adverse effects
- If a writer and some other process(reader/writer) access shared object simultaneously chaos may ensue

Requirements

- Writers must have exclusive access to the shared object
- R-W : not permitted
- W-R : not permitted
- W-W : not permitted
- R-R : permitted

Concurrency and Synchronization Issues

1. Race Condition:

- If a reader or writer does not acquire locks correctly before accessing the shared resource, a race condition can occur, leading to inconsistent or corrupted data.
- For example, if a reader is reading data while a writer is modifying it, the reader may get inconsistent data.

2. Mutual Exclusion:

- Writers need mutual exclusion to prevent simultaneous write operations, which could lead to data corruption.
- Readers don't need mutual exclusion from each other, but they need to ensure that no writer is modifying the resource when they are reading.

3. Deadlock:

- Deadlock can occur if readers and writers are blocked waiting for each other in a circular dependency.
- For example, a writer is waiting for readers to finish, while readers are waiting for the writer to release the resource.

Concurrency and Synchronization Issues

4. Starvation:

- Starvation occurs when one group (readers or writers) is continually given priority, preventing the other group from accessing the resource.
- Writer Starvation: If the system gives continuous priority to readers, writers may starve (never get a chance to write). This is known as Reader Priority.
- Reader Starvation: If writers are given priority over readers, readers may starve if new writers keep coming. This is called Writer Priority.

5. Fairness:

- A fair solution must ensure that both readers and writers get a reasonable opportunity to access the resource.
- Fairness is about avoiding starvation and ensuring that neither readers nor writers are indefinitely delayed.

Example- a library catalog

- Ordinary users of the library read the catalog to locate a book.
- One or more librarians are able to update the catalog.
- In the general solution-
 - every access to the catalog would be treated as a critical section, and
 - users would be forced to read the catalog one at a time.
 - would clearly impose intolerable delays.
- At the same time, it is important to prevent writers from interfering with each other
- it is also required to prevent reading while writing is in progress to prevent the access of inconsistent information.

Producer/Consumer vs Readers/Writers

- Can we take a single writer (the producer) and a single reader (the consumer)?
- **The answer is no.**
- The producer is not just a writer.
- It must read queue pointers to determine where to write the next item,
- it must determine if the buffer is full.
- Similarly, the consumer is not just a reader,
- it must adjust the queue pointers to show that it has removed a unit from the buffer.
- Reader must maintain queue pointers to determine where to read the next item

Readers-Writers Problem

- Several variations– all involving priorities
- Reader priority Problem-
 - No reader will be kept waiting unless a writer has already obtained permission to use the shared object.
 - No reader should wait for other readers to finish simply because a writer is waiting
- Writer priority Problem-
 - Once a writer is ready , that writer performs its write as soon as possible
 - If a writer is waiting to access the object, no new readers may start reading
- Fair policy reader-writer problem

R-W Problem

- The structure of a reader process

```
do {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);
    signal(mutex);

    ...
    /* reading is performed */
    ...

    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(rw_mutex);
    signal(mutex);
} while (true);
```

Reader

- Initially readcount=0
- First Reader R1 tries to enter
- wait mutex so mutex=0
- readcount=1
- if readcount is one i.e. First reader
- wait rw_mutex so rw_mutex=0
- readcount updation finished so signal mutex, mutex=1
- Reader enters CS
- “Reader R1 is reading”

Writer

- Now Writer tries to enter
- It executes his code
- wait(rw_mutex)
- gets trapped in infinite loop as rw_mutex was already 0 (reader priority)
- Writer cannot enter CS

```
do{
    wait(rw_mutex);
    ...
    /* writing is performed */
    ...
    signal(rw_mutex);
} while (true);
```

W-R Problem

- The structure of a reader process

```
do {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);
    signal(mutex);

    ...
    /* reading is performed */
    ...

    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(rw_mutex);
    signal(mutex);
} while (true);
```

Writer1

- Initially rw_mutex=1
- First Writer W1 tries to enter CS
- wait operation
- rw_mutex becomes 0
- Writer W1 enters CS

Reader1

- Initially mutex=1, readcount=0
- Now reader R1 tries to enter CS
- wait mutex
- mutex=0
- readcount=1
- In if section,
- wait(rw_mutex)
- gets trapped in infinite loop (Writer priority)
- Reader Cannot enter CS

```
do{
    wait(rw_mutex);
    ...
    /* writing is performed */
    ...
    signal(rw_mutex);
} while (true);
```

W-W Problem

- The structure of a reader process

```
do {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);
    signal(mutex);

    /* reading is performed */

    ...

    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(rw_mutex);
    signal(mutex);
} while (true);
```

Writer1

- Initially rw_mutex=1
- First Writer W1 tries to enter CS
- wait operation
- rw_mutex becomes 0
- Writer W1 enters CS

Writer2

- Another Writer W2 tries to enter CS
- wait operation
- rw_mutex is 0
- Thus trapped in Infinite loop
- Writer Cannot enter CS

```
do{
    wait(rw_mutex);
    ...
    /* writing is performed */
    ...
    signal(rw_mutex);
} while (true);
```

R-R Problem

- The structure of a reader process

```
do {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);
    signal(mutex);

    ...
    /* reading is performed */

    ...
    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(rw_mutex);
    signal(mutex);
} while (true);
```

Reader1

- Initially readcount=0
- First Reader R1 tries to enter
- wait mutex so mutex=0
- readcount=1
- if readcount is one i.e. First reader
- wait rw_mutex so rw_mutex=0
- readcount updation finished so signal mutex,
- mutex=1
- Reader enters CS

Reader2

- Reader R2 tries to enter CS
- wait mutex so mutex=0
- readcount=2
- Does not enter If section
- signal mutex
- mutex=1
- Reader R2 Enters CS

```
do{
    wait(rw_mutex);
    ...
    /* writing is performed */
    ...
    signal(rw_mutex);
} while (true);
```

Mutex solution for fair readers-writers problem

```
//Initialize:
mutex = CreateMutex()    // Mutex for synchronizing read_count
wrt = CreateSemaphore(1)  // Semaphore for writer access
read_count = 0           // Number of active readers

//Writer
function Writer(){
    while true:
        Wait(wrt)        // Wait for exclusive access to write

        // Critical Section (Writing)
        Write to shared resource // Perform write operation

        Signal(wrt)      // Release exclusive access
    }//writer

//Reader
function Reader(){
    while true:
        Lock(mutex)      // Lock to update read_count
        read_count = read_count + 1 // Increment the number of readers
        if read_count == 1: // If this is the first reader
            Wait(wrt)      // Wait for writer access (lock for writing)
        Unlock(mutex)     // Unlock after updating read_count

        // Critical Section (Reading)
        Read from shared resource // Perform read operation

        Lock(mutex)      // Lock to update read_count
        read_count = read_count - 1 // Decrement the number of readers
        if read_count == 0: // If this is the last reader
            Signal(wrt)    // Signal that writers can access
        Unlock(mutex)     // Unlock after updating read_count
    }//reader
```

Analysis of Fairness

- **No starvation:** The solution avoids both reader and writer starvation. Writers wait for ongoing readers to finish but will not be indefinitely delayed because once the last reader finishes, the writer gets exclusive access.
- **Mutual Exclusion:** Writers always get exclusive access to the critical section.
- **Reader Priority:** Readers are given priority in the sense that multiple readers can access the resource simultaneously, while writers have to wait if there are any active readers. However, once a writer starts, no new readers can access the resource until the writer finishes.
- **Possible Edge Cases:**
 - **Writer starvation in case of frequent readers:** While writers aren't starved in this implementation, there is a slight delay for writers if readers continuously arrive. However, this delay is minimal due to the condition that only when `read_count == 0` can a writer access the resource.

Solution for Fair Readers-Writers Problem Using Semaphores

Initialize:

```
mutex = CreateSemaphore(1)    // Mutex to protect read_count
wrt = CreateSemaphore(1)      // Semaphore for writer access
read_count = 0                // Number of active readers
read_lock = CreateSemaphore(1) // Semaphore for reader access
```

//Writer

```
function Writer(){
```

```
    while true:
```

```
        Wait(wrt)                // Wait for exclusive access to write
```

```
        // Critical Section (Writing)
```

```
        Write to shared resource    // Perform write operation
```

```
        Signal(wrt)                // Release exclusive access
```

```
    }//writer
```

//Reader

```
function Reader(){
```

```
    while true:
```

```
        Wait(read_lock)            // Wait to enter reader section
```

```
        Wait(mutex)                // Lock to update read_count
```

```
        read_count = read_count + 1 // Increment reader count
```

```
        if read_count == 1:         // If this is the first reader
```

```
            Wait(wrt)                // Wait for writer access
```

```
        Signal(mutex)              // Unlock after updating read_count
```

```
        Signal(read_lock)           // Allow other readers to enter
```

```
        // Critical Section (Reading)
```

```
        Read from shared resource    // Perform read operation
```

```
        Wait(mutex)                // Lock to update read_count
```

```
        read_count = read_count - 1 // Decrement reader count
```

```
        if read_count == 0:         // If this is the last reader
```

```
            Signal(wrt)              // Signal that writers can access
```

```
        Signal(mutex)              // Unlock after updating read_count
```

```
    }//reader
```

Fairness & Performance Summary

- **Reader-Reader Fairness:** Multiple readers can read the shared resource concurrently. The use of `read_lock` prevents contention among readers when they try to increment or decrement `read_count`. The design ensures fair access for all readers without starving any particular reader.
- **Reader-Writer Fairness:** Writers are not starved, as they will eventually access the shared resource once all current readers finish. The use of `wrt` ensures that new readers cannot start reading while a writer is waiting, which promotes fairness between readers and writers.
- **Writer-Writer Fairness:** Writers access the shared resource one at a time in a fair manner due to the `wrt` semaphore. No writer is starved, and they get serialized access.
- **Writer-Reader Fairness:** Writers have to wait for readers to finish, and once they begin writing, no readers can enter until the writer finishes. This promotes a fair balance between readers and writers.

Fair share with semaphores- Starvation and Livelock Considerations

- **No Starvation:** Both readers and writers get a fair share of access to the shared resource. Readers can access concurrently, and writers get their exclusive access once readers are done. The use of semaphores ensures that no group (readers or writers) will be indefinitely delayed.
- **No Livelock:** The solution prevents livelock since the read_lock semaphore ensures that readers can't endlessly preempt writers. Writers are guaranteed eventual access after all readers finish.

Solution to readers-writers : comparison

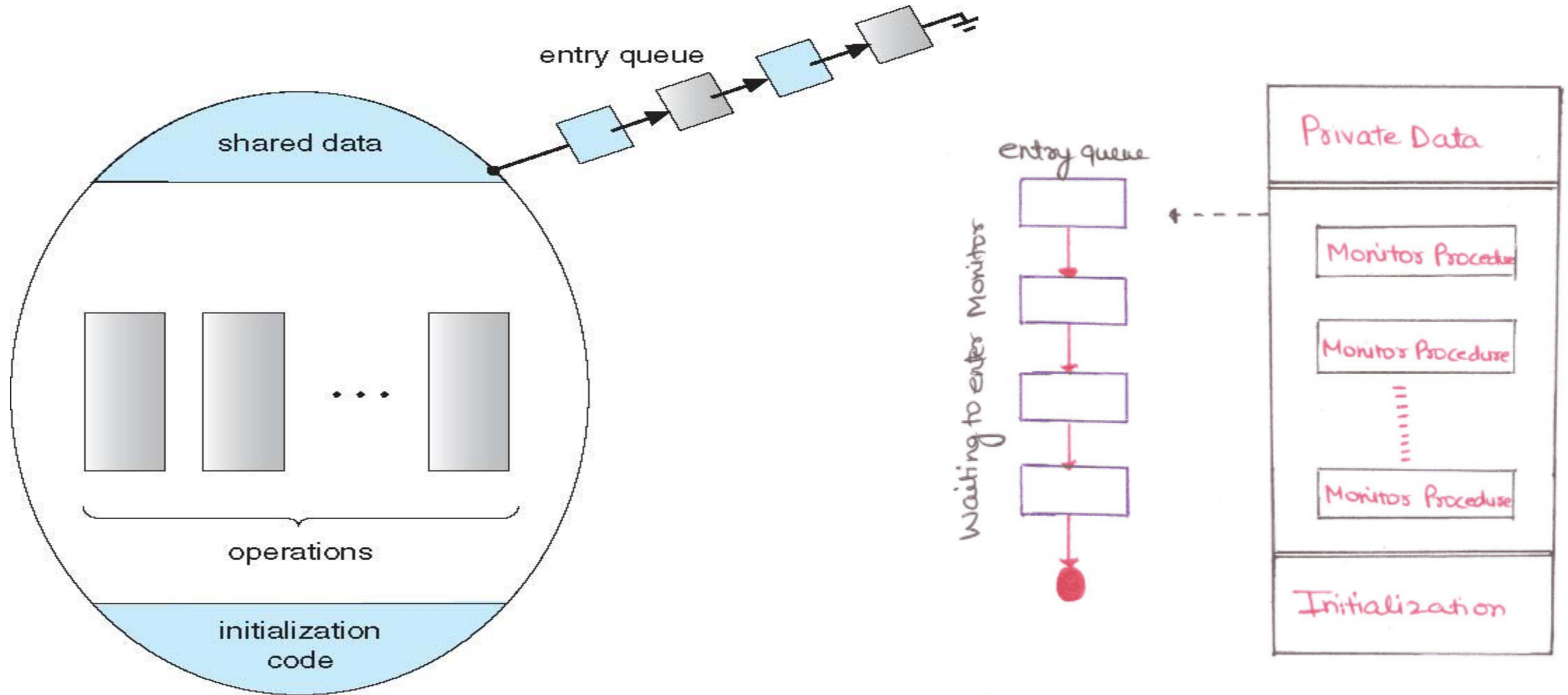
Fairness for Writers	Writers may starve in a read-heavy environment.	Fair, as read_lock prevents new readers when a writer is waiting.	Writers get exclusive access but may wait behind readers.
Mutual Exclusion	Handled using mutex for writers, but readers share access.	Handled with wrt for writers, fine-tuned with semaphores.	Managed automatically via monitor's internal locks.
Complexity	Simple to implement, especially for readers.	More complex due to multiple semaphores for fairness.	Simplified due to high-level abstraction, fewer details.
Performance	Efficient for concurrent readers, but may starve writers.	Good performance but with extra overhead for managing semaphores.	Performance may vary depending on monitor implementation.
Starvation/ Livelock	Writer starvation possible.	No starvation for either group if semaphores are properly managed.	Can be designed to avoid starvation using condition variables.

Monitors

Monitors

- A high-level synchronization construct
- *Set of programmer defined operators*
- *Declaration of variables*
 - whose *value define the state* of an instance of the type
- *Bodies of procedures*
 - that *implement operations* on the type

Schematic view of a Monitor



Syntax of Monitors

```
monitor monitor-name
{
    // shared variable declarations
    procedure P1 (...) { ... }

    procedure Pn (...) {.....}

    Initialization code (...) { ... }
}
}
```

Monitors

- *Abstract data type*, internal variables only accessible by code within the procedure
- Procedure defined within the monitor can access only those variables
 - declared locally within the monitor and
 - its formal parameters.

```
monitor monitor-name
{
    // shared variable declarations
    procedure P1 (...) { ... }

    procedure Pn (...) {.....}

    Initialization code (...) { ... }
}
```

Monitors

- It is the collection of condition variables and procedures combined together in a special kind of module or a package.
- The processes running outside the monitor can't access the internal variable of the monitor but can call procedures of the monitor.
- Only one process at a time can execute code inside monitors.

Monitors

- Monitor ensures that
 - Only one process may be active within the monitor at a time
- Programmer does not need to
 - code the synchronization constraint explicitly
- But not powerful enough to model some synchronization schemes
- Need to define additional condition construct

Condition Variables

- Programmer can define **one or more condition variables**

condition x, y;

- **Only Two operations** are allowed on a condition variable:

- **x.wait()**
- **x.signal()**

```
Monitor Demo //Name of Monitor
{
variables;
condition variables;

procedure p1 {...}
prodecure p2 {...}

}
```

Syntax of Monitor

Condition Variables

- **`x.wait()`** –
 - Process performing wait operation on any condition variable are suspended.
 - suspended until **`x.signal()`**
 - The suspended processes are placed in **block queue of that condition variable**.
- **`x.signal()`** –
 - When a process performs signal operation on condition variable, **one of the blocked processes is given chance**.
 - resumes one of processes (if any) that invoked **`x.wait()`**
 - If no **process is suspended**, then it has no effect on the variable
 - State of x, As if the operation was never executed
 - In contrast to semaphore, state of semaphore always gets affected

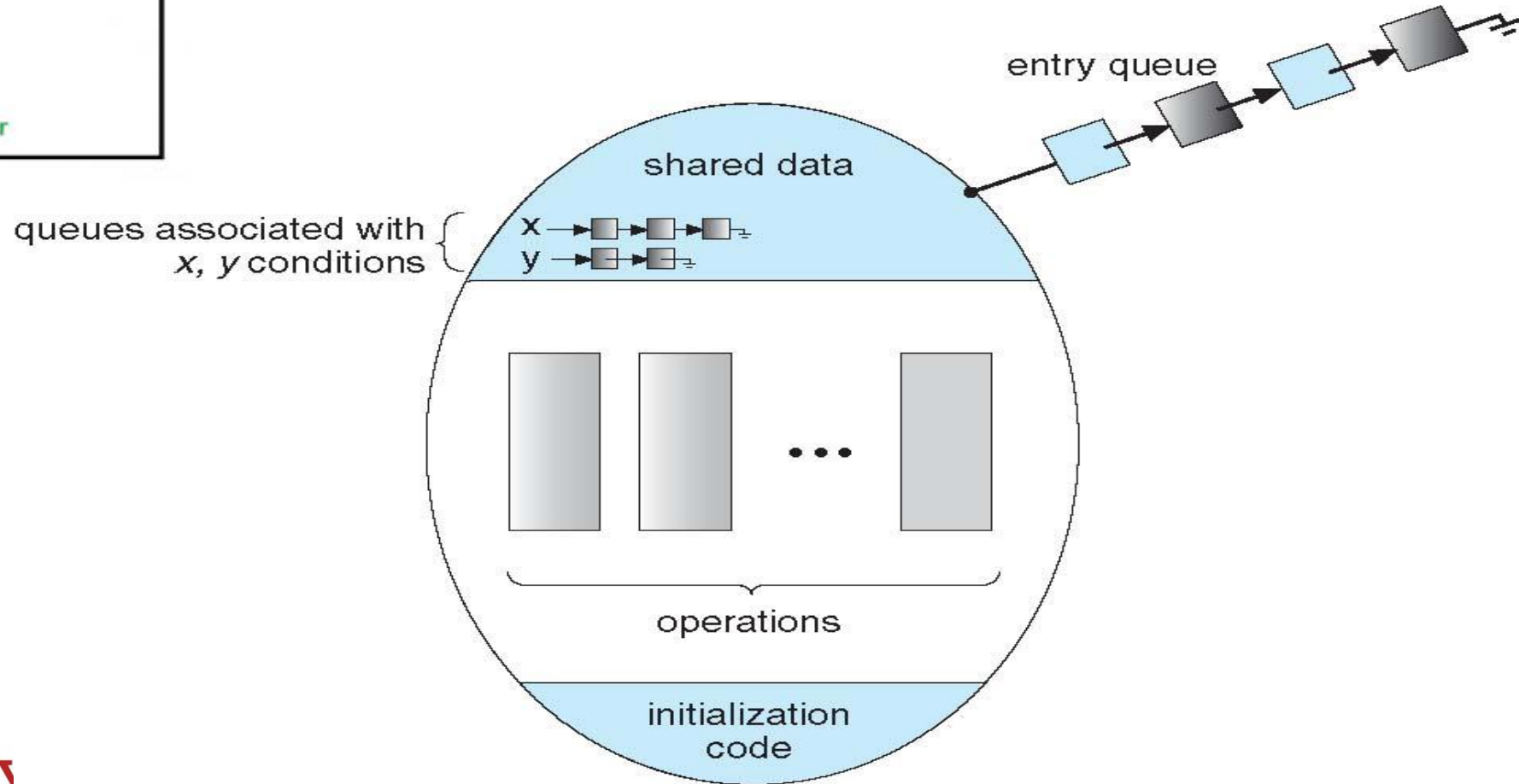
Monitor with Condition Variables

```
Monitor Demo //Name of Monitor
{
  variables;
  condition variables;

  procedure p1 {...}
  procedure p2 {...}

}
```

Syntax of Monitor



Condition Variables Choices

- If process P invokes **`x.signal()`** , and process Q is suspended in **`x.wait()`** ,
 - Suspended process Q associated with condition x is invoked
 - what should happen next?
- Both Q and P cannot execute in parallel.
 - If Q is resumed, then P must wait
 - Otherwise both P and Q will be active simultaneously within the monitor

Condition Variables Choices

- Options include
 - **Signal and wait** – P waits until Q either leaves the monitor or it waits for another condition
 - **Signal and continue** – Q waits until P either leaves the monitor or it waits for another condition

Condition Variables Choices

- Since P was already executing in the monitor,
 - Option 2 seems more reasonable
 - However, If P continues, the logical condition for which Q was waiting may no longer hold by the time Q is resumed
- Both have pros and cons – language implementer can decide
- Monitors implemented in Concurrent Pascal compromise
 - P executing signal immediately leaves the monitor, Q is resumed
- Implemented in other languages including Mesa, C#, Java

```

monitor <MonitorName> {

    // Shared resources (variables, data structures) are declared as private
    private:
        <shared variables>;

    // Condition variables to manage synchronization
    condition <ConditionVariable1>;
    condition <ConditionVariable2>;
    // More condition variables as needed

    // Entry procedures (only one thread is allowed to execute one of these at a time)
    public:
        procedure <Operation1>(parameters) {
            // Synchronization logic
            // Access shared resources
            // Possibly wait or signal condition variables
        }

        procedure <Operation2>(parameters) {
            // Synchronization logic
            // Access shared resources
            // Possibly wait or signal condition variables
        }

    // Initialization procedure to initialize shared resources
    procedure Init() {
        // Initialize shared resources
    }

}

```

Monitor control structure

```

// Monitor Structure
monitor BoundedBuffer {
    condition not_full; // Condition variable for buffer not being full
    condition not_empty; // Condition variable for buffer not being empty
    item buffer[MAX_SIZE]; // The shared buffer
    int in = 0, out = 0, count = 0; // Pointers and count for the circular buffer

    // Add an item to the buffer (Producer)
    procedure add_item(item x) {
        if (count == MAX_SIZE) {
            wait(not_full); // Wait until there is space in the buffer
        }
        buffer[in] = x; // Add the item to the buffer
        in = (in + 1) % MAX_SIZE; // Circular buffer
        count++; // Update the count

        signal(not_empty); // Signal that buffer is no longer empty
    } // procedure add_item

    // Remove an item from the buffer (Consumer)
    procedure remove_item() -> item {
        if (count == 0) {
            wait(not_empty); // Wait until there is an item in the buffer
        }
        item x = buffer[out]; // Remove the item from the buffer
        out = (out + 1) % MAX_SIZE; // Circular buffer
        count--; // Update the count
        signal(not_full); // Signal that buffer is no longer full
        return x;
    } // procedure remove_item
} // monitor

```

```

// Producer
function producer() {
    while (true) {
        item = produce_item(); // Produce an item

        // Add the item to the shared buffer
        BoundedBuffer.add_item(item);
    }
} // producer

// Consumer \
function consumer() {
    while (true) {
        // Remove the item from the shared buffer
        item = BoundedBuffer.remove_item();

        consume_item(item); // Consume the item
    }
} // consumer

```

Bounded buffer PC using Monitors

Explanation of Solution

1. Monitor:

- The monitor encapsulates the shared buffer (`buffer[]`), the `in` and `out` pointers for the circular buffer, and the `count` variable to track the number of items.
- Mutual exclusion is inherently managed by the monitor.
- Only one thread (either a producer or a consumer) can execute a monitor procedure at a time, ensuring safe concurrent access.

2. Condition Variables:

- `not_full`: The producer will wait on this condition if the buffer is full. When the consumer removes an item and signals the producer, the producer can add a new item to the buffer.
- `not_empty`: The consumer will wait on this condition if the buffer is empty. When the producer adds an item and signals the consumer, the consumer can remove the item from the buffer.

3. Producer:

- When the producer wants to add an item, it first checks if the buffer is full (i.e., `count == MAX_SIZE`). If the buffer is full, the producer waits on the `not_full` condition.
- After the consumer consumes an item and signals the producer (via `signal(not_full)`), the producer resumes, adds the item to the buffer, and then signals the consumer (via `signal(not_empty)`) that the buffer is no longer empty.

4. Consumer:

- When the consumer wants to remove an item, it first checks if the buffer is empty (i.e., `count == 0`). If the buffer is empty, the consumer waits on the `not_empty` condition.
- After the producer produces an item and signals the consumer (via `signal(not_empty)`), the consumer resumes, removes the item from the buffer, and then signals the producer (via `signal(not_full)`) that the buffer is no longer full.

Analysis of Solution

1. Mutual Exclusion:

- The monitor guarantees that only one thread can be inside the `add\_item` or `remove\_item` procedures at a time.
- This ensures mutual exclusion, as it prevents race conditions when accessing the shared buffer. The `mutex` is implicitly managed by the monitor, so no additional locking mechanism is required.

2. Condition Synchronization:

- Blocking: The producer is blocked if the buffer is full, and the consumer is blocked if the buffer is empty. Condition variables (`not\_full` and `not\_empty`) ensure that producers and consumers only proceed when it is safe to do so.
- Signaling: Producers signal consumers when an item is added (buffer not empty), and consumers signal producers when an item is consumed (buffer not full). This ensures that producers and consumers are awakened when their conditions change.

3. Deadlock-Free:

- The solution is deadlock-free because producers and consumers only wait when necessary (on full or empty conditions). Once the buffer state changes (due to item addition or removal), the corresponding waiting process is signaled to continue.

4. Fairness and Starvation:

- Fairness: No producer or consumer will starve, as each process waiting on the condition variable (`not\_full` or `not\_empty`) will be signaled as soon as the required condition is satisfied.
- No Starvation: Every producer will eventually add its item to the buffer when space is available, and every consumer will eventually consume an item when one is available

Monitors

- **Advantages of Monitor:**

- Make parallel programming easier and
- less error prone than using techniques such as semaphore.

- **Disadvantages of Monitor:**

- Monitors have to be implemented as part of the programming language .
- The compiler must generate code for them.
- This gives the compiler the additional burden of having to know what operating system facilities are available to control access to critical sections in concurrent processes.
- Some languages that do support monitors are Java,C#,Visual Basic,Ada and concurrent Euclid.

Monitor Solution to Dining Philosophers

- Deadlock free solution to Dining Philosophers Problem
- To distinguish amongst the 3 states in which the philosopher may be-
 - **enum { THINKING; HUNGRY, EATING) state [5] ;**
- Philosopher i can set the variable state [i] = EATING only
 - **if her two neighbors are not eating**
 - **(state [(i +4) % 5] != EATING) and (state [(i +1) % 5] != EATING)**

Monitor Solution to Dining Philosophers

- Also need to declare
 - condition self[5];
 - in which i^{th} philosopher **can delay herself when she is hungry but is unable to obtain the chopsticks she needs.**
- Distribution of chopsticks is controlled by the **monitor DiningPhilosophers**

Monitor Solution to Dining Philosophers

```
monitor DiningPhilosophers
```

```
{
```

```
    enum { THINKING; HUNGRY, EATING) state [5] ;
```

```
    condition self [5];
```

```
    void pickup (int i) {
```

```
        state[i] = HUNGRY;
```

```
        test(i);
```

```
        if (state[i] != EATING)
```

```
            self[i].wait;
```

```
}
```

Pickup chopsticks

- If unable to eat, wait to be signaled
- Philosopher can delay herself when she is hungry but is unable to obtain the chopsticks she needs.

Monitor Solution to Dining Philosophers

```
monitor DiningPhilosophers  
{
```

```
void putdown (int i) {
```

```
    state[i] = THINKING;
```

```
        // test left and right neighbors
```

```
    test((i + 4) % 5);
```

```
    test((i + 1) % 5);
```

Put down chopsticks

if right neighbor $R=(i+1)\%5$ is hungry and
both of R's neighbors are not eating,
set R's state to eating and wake up neighbour R by signaling

Monitor Solution to Dining Philosophers

```
void test (int i) {  
    if ((state[(i + 4) % 5] != EATING) &&  
        (state[i] == HUNGRY) &&  
        (state[(i + 1) % 5] != EATING) ) {  
        state[i] = EATING ;  
        self[i].signal () ;  
    }  
}
```



- If her two neighbors are not eating and she is hungry
 - I.E. if my left and right neighbors are not eating
- Set her state as eating

```
initialization_code() {  
    for (int i = 0; i < 5; i++)  
        state[i] = THINKING;  
}
```

- signal() has no effect during Pickup(),
- but is important to wake up waiting hungry philosophers during Putdown()

Solution to Dining Philosophers

- Each philosopher i invokes the operations `pickup()` and `putdown()` in the following sequence:

`DiningPhilosophers.pickup(i) ;`

EAT

`DiningPhilosophers.putdown(i) ;`

- No deadlock, but starvation is possible

Solution to Dining Philosophers

- Each philosopher, before starting to eat, must invoke the operation pickup().
- This act may result in the suspension of the philosopher process.
- After the successful completion of the operation, the philosopher may eat.
- After eating, philosopher invokes putdown() and start to think

Solution to Dining Philosophers

- Execution of Pickup(), Putdown() and test() are all mutually exclusive, i.e. only one at a time can be executing
- No 2 neighbors are eating simultaneously
- So No deadlock, but starvation is possible